AI/ML Infrastructure from Silicon to Scale

The Principal Performance Architect's Complete Guide to Designing, Optimizing, and Operating AI Systems — from GPU Microarchitecture to Hundred-Thousand-GPU Clusters

---

Author: Venkat Vinjam
Role: AI/ML Performance Architect · AMD | Ex-Intel
Edition: First Edition · 2026
Coverage: 6 Parts · 18 Chapters · 3 Appendices · 450+ Pages
Hardware Generation: NVIDIA H100/H200/B100/GB200, AMD MI300X/MI350, Intel Gaudi 3 (current as of 2026)
Software Stack: CUDA 12.x · PyTorch 2.x · Triton · vLLM · Megatron-LM · NCCL 2.27 · SLURM · Kubernetes

© 2026 Venkat Vinjam. All rights reserved.
No part of this publication may be reproduced without written permission.

> *"Performance engineering is not about making one function faster.*
> *It is about understanding the whole system — from transistors to tokens —*
> *and knowing exactly where the bottleneck lives before you touch anything."*
> — V. Vinjam, Chapter 1

Note on Accuracy and Currency

Hardware specifications, software APIs, and vendor product details change rapidly in the AI infrastructure field. All quantitative claims in this book carry **confidence labels** so you always know how to weight the information:

Label	Meaning
`[SHIPPED]`	Verified shipping hardware or software — measured numbers from public benchmarks
`[ANNOUNCED]`	Vendor-disclosed but not yet shipping as of publication — treat as directional
`[ESTIMATED]`	Calculated or inferred from first principles or partial data — methodology shown
`[REPRESENTATIVE]`	Illustrative example using plausible values — not a specific hardware claim
`[ENV-SPECIFIC]`	Cluster-configuration or framework-dependent — your results will vary

Code blocks carry equivalent labels. When hardware specifications matter for a decision, verify against current vendor documentation. The *methods* in this book are stable across generations; the *numbers* require annual refresh.

Preface

Who This Book Is For

This book was written for a specific engineer at a specific inflection point in their career.

You have spent years — perhaps a decade or more — mastering performance engineering in systems you understand deeply: CPU microarchitecture, HPC cluster optimization, cloud infrastructure, embedded systems, memory subsystems, or networking stacks. You are proficient at profiling, skilled at bottleneck analysis, and you think in tradeoffs. You are, by every measure, a strong senior or principal engineer.

But the field you work in has shifted under your feet. The dominant computing problem of the next decade is not web-scale distributed systems or HPC simulation. It is training and serving large language models — and the principal-level positions at the companies building those systems require a very specific combination of skills: GPU architecture depth, distributed systems reasoning, numerical methods intuition, and the ability to communicate quantitative tradeoffs to hardware teams, software teams, and product leadership simultaneously.

This book is for engineers making that transition. It assumes you can already think in systems. It will teach you the domain.

What This Book Is Not

This is not an introduction to machine learning. It does not explain what a neural network is to someone who has never seen one, and it does not walk you through training your first model. There are excellent resources for that — this is not one of them.

This is not a GPU programming tutorial. You will not finish this book and be ready to write production CUDA kernels from scratch. Chapter 7 gives you the mental model and the vocabulary to reason about kernel performance — but the engineering of high-performance CUDA is a craft that takes years, and this book respects that boundary.

This is not a paper survey. Relevant research papers are cited and summarized throughout, but the goal is always to distill the engineering insight, not catalog the literature.

What this book *is*: a unified, quantitative, practitioner-grade reference that connects GPU silicon to cluster-level AI infrastructure. Silicon to scale — literally. The analytical thread running through every chapter is the same: arithmetic intensity, memory bandwidth, and communication volume. These three quantities are the grammar of AI performance engineering. Master them, and every new hardware generation, every new model architecture, every new collective communication algorithm becomes legible from first principles.

What Makes This Book Different

There are research papers on distributed training and inference systems. There are GPU programming guides. There are blog posts on vLLM and Megatron-LM. What did not exist before this book is a single reference that unifies all of these into a coherent, quantitative, practitioner-grade guide from hardware silicon to cluster-level optimization.

The structural choices that make this book distinctive:

Silicon to Scale, Literally. Chapter 1 begins with the GPU execution model at the SM and warp level. The final chapters operate at the 50,000-GPU cluster level. The analytical thread connecting these two scales — roofline analysis, arithmetic intensity, communication-to-compute ratios — runs consistently throughout.

Formulas, Diagrams, and Production Numbers. Every performance claim is grounded in math. Every architectural pattern is illustrated. Every worked example uses real hardware specifications (`[SHIPPED]` H100, MI300X, GB200) with the computation shown explicitly. You can reproduce every result on your own hardware.

The Confidence Label System. AI hardware evolves faster than any book can track. Every quantitative hardware claim carries a label so you always know whether you are reading a measured fact, a vendor disclosure, or an engineering estimate.

Vertical Integration. A chapter on inference serving explains not just how vLLM works, but *why* PagedAttention was necessary given DRAM capacity constraints — which connects back to the KV cache memory model — which connects back to the transformer attention arithmetic. The book teaches you to reason across layers, not just within them.

Interview Preparation Built In. Each chapter ends with review questions calibrated to principal-level interviews. A dedicated interview prep path is provided in "How to Read This Book." The system design walkthroughs in Chapter 18 are the most detailed available outside of closed interview preparation materials.

The Central Identity This Book Builds

The strongest version of the AI/ML Principal Performance Architect identity is not *"I optimized a CUDA kernel."* It is:

> *"I understand the AI system as a whole — from the transistor physics of the GPU execution units to the TCO implications of a rack-level power budget — and I can reason quantitatively about tradeoffs across every layer simultaneously, then communicate those tradeoffs in terms that move hardware and software decisions forward."*

That is the engineer this book was written for. And, after working through it carefully, that is the engineer you will be equipped to become.

— Venkat Vinjam, April 2026

How to Use This Book

A practical navigation guide for four reader types

This book was written as a single coherent work. You can read it front-to-back — it builds systematically, each chapter scaffolded on the previous. But most engineers who pick up this book are not reading for pleasure. They are reading with a specific goal, a specific timeline, and a specific gap to close. This section gives you a direct path to your goal without requiring you to read sequentially.

The four paths below reflect the four most common reader profiles. Find yours, follow the sequence, and treat everything else as reference material to return to once the core path is complete.

Path 1 — The Interview-Focused Reader

Situation: You have an interview at a company building or operating AI/ML infrastructure at scale — NVIDIA, Google, Meta, Microsoft, Amazon, Apple, or a frontier lab. You have two weeks.

Every hour counts.

The Constraint: You need enough depth to answer system design questions and behavioral questions at principal or senior staff level. You do not need to become a kernel engineer in two weeks. You need to be dangerous at the analytical and architectural level.

Daily Discipline: 2 hours per day, minimum. Focus every session on formulas and mental math — not prose. If you cannot derive the roofline bound for a GEMM from first principles without notes by Day 3, go back to Chapter 1. Numbers beat descriptions in every technical interview.

```
WEEK 1 — Build the analytical foundation and mental models

  Day 1-2:  Chapter 1   — Roofline, arithmetic intensity, MFU.
                          Memorize the three performance regimes.
                          Derive the ridge point formula from scratch.

  Day 3-4:  Chapter 3A  — GPU/accelerator architecture.
                          Know H100 SXM specs cold: 989 TFLOPS BF16,
                          3.35 TB/s HBM3, 900 GB/s NVLink.
                          Understand SXM vs. PCIe bandwidth implications.

  Day 5-6:  Chapter 6   — Inference systems.
                          Prefill vs. decode regimes. Continuous batching.
                          TTFT vs. TPOT tradeoffs. Throughput-latency curves.

  Day 7:    Chapter 11  — KV cache as virtual memory.
                          PagedAttention mental model.
                          KV cache memory sizing formula — derive it cold.

WEEK 2 — Scale, depth, and synthesis

  Day 8-9:  Chapter 10  — Distributed training.
                          4D parallelism: TP, PP, DP, SP — when each applies.
                          Gradient bucketing. Pipeline bubble math.
                          ZeRO stages: what each shards, memory formula.

  Day 10-11: Chapter 14 — Networking and communication.
                          AllReduce algorithms: ring, tree, recursive halving.
                          Bandwidth vs. latency regime transitions.
                          NCCL environment variable tuning — know the top 5.

  Day 12-13: Appendix A — Hardware reference tables.
                          Commit the top 3 GPU specs to memory.
                          Know the InfiniBand bandwidth numbers.
                          Run the TCO mental math for a 1,024-GPU cluster.

  Day 14:   Chapter 18  — Principal career and interview playbook.
                          Read the system design walkthroughs aloud.
                          Practice the STAR+Impact stories with your own data.
                          Run one full mock design session (45 minutes, timed).
```

The single most important thing to do before the interview: Run the LLM inference serving design (§18.10) and the distributed training design (§18.11) as mock interviews. Set a timer for 45

minutes each. Do not look at the chapter while you go through it. Then compare. Fix the gaps.

Path 2 — The Practicing Inference Engineer

Situation: You are actively building or operating LLM inference systems — vLLM, TensorRT-LLM, SGLang, or a custom serving stack. You understand the basics. You want to optimize aggressively and reason about tradeoffs you currently feel rather than calculate.

The Constraint: You need depth on the right topics, not breadth across all topics. Start with the inference stack, then pull in supporting depth (memory, quantization, telemetry) as you need it.

Reference Practice: Keep Appendix A open at all times. Every time you see a number in a chapter, verify it against the hardware spec. Build intuition about which operations are near the hardware limit on your specific GPU.

```
START (Weeks 1–3): Core inference depth

  Chapter 6   — Inference systems architecture.
                Read every section. The prefill/decode regime analysis
                is the foundation of everything else.

  Chapter 11  — KV cache engineering.
                PagedAttention deep dive. Fragmentation analysis.
                Prefix caching. Memory sizing at production scale.

  Chapter 8   — Quantization.
                INT8, FP8, GPTQ, AWQ. When quantization helps and
                when it does not. The deployment decision framework (§8.8).

DEPTH (Weeks 4–6): Supporting systems

  Chapter 7   — GPU kernels.
                FlashAttention mechanics. Fused kernels.
                How to read Nsight Compute output productively.

  Chapter 9   — Operator fusion and compiler optimization.
                torch.compile, Triton, kernel fusion candidates.
                Why fused kernels exist and what they actually save.

  Chapter 17  — Telemetry, observability, and profiling.
                Nsight Systems for system-level timeline.
                DCGM for fleet health. Prometheus/Grafana stack.

SCALE (Weeks 7–8): Multi-GPU and cost

  Chapter 14  — Networking.
                Tensor parallel communication patterns.
                NCCL tuning for inference.

  Chapter 5   — Thermal and TCO.
                Power limits under sustained inference load.
                $/token as the primary cost metric.
```

The one project to complete: Profile a real vLLM deployment with Nsight Systems during a mixed prefill-decode workload. Identify where time is actually going — prefill compute, decode memory bandwidth, or scheduling overhead. The answer will surprise you.

Path 3 — The Training Infrastructure Architect

Situation: You are designing, procuring, or operating large-scale GPU training clusters. You need to reason about 4D parallelism, network topology, storage pipelines, fault tolerance, and cluster TCO simultaneously.

The Constraint: You need the full architecture and infrastructure picture. Start with foundations, then build up to cluster-scale reasoning.

```
FOUNDATION (Weeks 1–2):

  Chapter 1   — Performance mindset. Roofline at cluster scale.
                MFU as the cluster-wide efficiency metric.

  Chapter 3A  — GPU architecture fundamentals.
                NVLink topology. SXM vs. PCIe for training.
                Why NVSwitch matters at 8-GPU and 256-GPU scale.

CORE SYSTEMS (Weeks 3–5):

  Chapter 10  — Training systems architecture at scale.
                4D parallelism decision tree.
                ZeRO memory analysis. Pipeline bubble math.
                Fault tolerance at 1,000+ GPU scale.

  Chapter 2   — Transformer architecture.
                Attention arithmetic and why it shapes parallelism decisions.
                GQA, MQA — memory implications for the KV cache.

INFRASTRUCTURE (Weeks 6–8):

  Chapter 14  — Networking and collective communication.
                InfiniBand HDR/NDR topology. ECMP and RDMA.
                AllReduce algorithm selection at scale.

  Chapter 15  — Storage, data pipelines, and I/O.
                Checkpoint storage sizing and frequency math.
                Data pipeline bottleneck analysis. Sequence packing.

  Chapter 16A — Cluster management and multi-tenant systems.
                SLURM vs. Kubernetes. Gang scheduling.
                Multi-tenant isolation and SLA enforcement.

  Chapter 17  — Telemetry and observability.
                Fleet-wide MFU tracking. Fail-slow GPU detection.
                Training job health monitoring at scale.

ARCHITECTURE SYNTHESIS (Weeks 9–10):

  Chapter 5   — Thermal, power, and TCO.
                Power delivery constraints at rack and cluster level.
                $/GPU-hour vs. $/FLOP vs. $/token cost modeling.
```

```
  Chapter 18  — Principal career (§18.9–18.11).
                System design: distributed LLM training cluster.
                Tradeoff framework for architecture decisions.
```

The one project to complete: Build a full TCO model for a 256-GPU H100 cluster vs. a 512-GPU A100 cluster for a target training workload (Chapter 5 provides the framework). Include power, cooling, capex, opex, and $/token. Present it to someone who will ask hard questions. This exercise forces integration of hardware specs, workload modeling, and cost reasoning in a way that reading alone cannot.

Path 4 — The Deep Learner (Full Stack Transition)

Situation: You are transitioning into AI/ML infrastructure from an adjacent domain — HPC, cloud systems, CPU performance engineering, embedded systems. You want to build the complete principal-level knowledge base from foundations to production systems.

The Constraint: You have time. Do not skip chapters. Do not treat this as a vocabulary-building exercise — build the skills, not just the terminology. The review questions at the end of each chapter are not optional on this path; they are the test.

```
FOLLOW THE BOOK IN ORDER: Parts I → II → III → IV → V → VI

  For each chapter:
    1. Read the chapter fully, taking notes on formulas and key decisions
    2. Verify every formula derivation by hand
    3. Answer all review questions without looking at the chapter
    4. Run the code examples if a GPU environment is available
    5. Build the personal reference sheet entry for this chapter

  After each Part:
    Review your personal reference sheet for the whole Part
    Run one mock system design question using concepts from that Part

REFERENCE: Chapter 18 for positioning and communication.
  §18.7 (written communication strategy) is essential for architects
  who influence hardware roadmaps and capital decisions. The ability
  to write a compelling architecture recommendation backed by
  benchmark data is a core architect competency — read this early
  and practice it throughout.
```

Quick Reference: Paths by Time Budget

Time Budget	Goal	Priority Chapters
2 hours	Emergency interview prep	Ch 1, 3A, 6 (§6.1–6.3), 11 (§11.1–11.2), 14 (§14.1–14.3), Appendix A
8 hours	Solid interview foundation	Ch 1, 2, 3A, 6, 10, 11, 14, Appendices A+B
Weekend	Thorough interview prep	Parts I–II, Ch 6, 7, 10, 11, 13, 14, Ch 18
4 weeks	Systematic study (one Part/week)	All 6 Parts in sequence, all review questions
Full read	Complete mastery (8–12 weeks)	All 18 chapters, all appendices, build one system design

---

The 12-Month Principal-Level Curriculum

For engineers who want to systematically build the full principal AI/ML infrastructure identity — not just prepare for an interview, but become the engineer who does not need to prepare.

How This Curriculum Is Structured

Each two-month block contains:

- **2 chapters** to read deeply — meaning: read, work the exercises, verify the formulas
- **1 hands-on project** to build and measure — reading without building is vocabulary without skill
- **3 papers** to read — the field's foundational literature, not optional
- **1 benchmark** to run — numbers you have measured personally are worth ten times numbers you have read

The curriculum is designed for 10–15 hours per week. A working engineer with a full-time job can run this in the background. An engineer between roles can compress some blocks.

Months 1–2: Foundations

Goal: You can explain roofline analysis, transformer architecture, and GPU memory hierarchy from first principles to another engineer, with correct numbers, without notes.

Chapters — Read Deeply:

- **Chapter 1** — AI/ML Performance Architecture Mindset. Read every section twice. Derive the ridge point formula five times until it is automatic. Build the roofline plot for H100 by hand using Appendix A numbers.
- **Chapter 2** — Transformer Architecture Deep Dive. Trace the FLOPs through a forward pass. Compute Q, K, V projection shapes for Llama-70B by hand. Understand why GEMM dominates and when attention becomes the bottleneck.

Hands-On Project: Implement and profile a minimal transformer. Build a pure PyTorch transformer forward pass from scratch — no `nn.Transformer`. Run it on a GPU. Profile with Nsight Systems. Identify the top three time consumers. Compute theoretical arithmetic intensity for the GEMM and attention operations and compare to measured throughput.

Papers to Read:

1. *Attention Is All You Need* (Vaswani et al., 2017) — the original transformer. Read for architecture clarity, not novelty.
2. *Roofline: An Insightful Visual Performance Model* (Williams et al., 2009) — the analytical foundation for Chapter 1.
3. *Efficient Large Scale Language Modeling with Mixtures of Experts* (Artetxe et al., 2021) — foreshadows Ch 13.

Benchmark to Run: `python -c "import torch; A=torch.randn(4096,4096).cuda(); B=torch.randn(4096,4096).cuda(); torch.cuda.synchronize(); import time; t=time.perf_counter(); C=A@B; torch.cuda.synchronize(); print(f'{2*4096**3/(time.perf_counter()-t)*1e-12:.1f} TFLOPS')"`. Compare to the H100 `[SHIPPED]` BF16 peak. Your GEMM MFU is the gap.

Months 3–4: Hardware Deep Dive

Goal: You can spec a GPU for a given workload from first principles, explain HBM3 bandwidth utilization, and reason about thermal and power constraints on sustained AI compute.

Chapters — Read Deeply:

- **Chapter 3A** — GPU and Accelerator Architecture Fundamentals. Know the SM execution model, tensor core operation, and NVLink topology. Be able to explain the H100 SXM vs. PCIe performance difference from first principles.
- **Chapter 4** — Memory Hierarchy and HBM Deep Dive. Understand HBM3 physical construction, bandwidth roofline, ECC overhead, and the memory access pattern implications for attention kernels.

Hands-On Project: Measure HBM bandwidth utilization on a vector operation, a GEMM, and an attention kernel. Use Nsight Compute (`ncu`) to get DRAM read/write bytes and compare to theoretical peak. Plot the three operations on the roofline. The pattern — bandwidth-bound, ridge-point, compute-bound — should be immediately visible.

Papers to Read:

1. *NVIDIA H100 Tensor Core GPU Architecture Whitepaper* (2022) — read the full whitepaper, not the press release.
2. *AMD CDNA3 Architecture Whitepaper: MI300X* (2023) — for comparative hardware reasoning.
3. *Demystifying GPU Microarchitecture through Microbenchmarking* (Mei and Chu, 2016) — how to measure what the spec sheet does not tell you.

Benchmark to Run: Run the NVIDIA `bandwidthTest` utility. Record HtoD, DtoH, and D2D bandwidth. Compare to the `[SHIPPED]` HBM3 spec. Compute utilization percentage. If you have access to MI300X, run the AMD equivalent and compare HBM3e bandwidth.

Months 5–6: Optimization Techniques

Goal: You can explain FlashAttention from algorithmic first principles, identify operator fusion opportunities in a PyTorch model, and select the right quantization strategy for a production deployment.

Chapters — Read Deeply:

- **Chapter 7** — GPU Kernels and CUDA Optimization. FlashAttention tiling mechanics, fused kernels, the memory access pattern taxonomy (coalesced, strided, random). How `ncu` counter values map to kernel-level bottlenecks.
- **Chapter 9** — Operator Fusion and Compiler Optimization. `torch.compile` internals, Triton kernel authoring, fusion candidate identification, compile failure modes.

Hands-On Project: Implement a fused LayerNorm + GeLU kernel in Triton. Compare to the unfused PyTorch baseline. Measure the memory bandwidth savings with `ncu`. The result should show 30–50% reduction in memory traffic for the fused operation. Document what you measured.

Papers to Read:

1. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness* (Dao et al., 2022) — derive the tiling math yourself after reading.
2. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning* (Dao, 2023) — the key algorithmic improvements.
3. Horace He, *Making Deep Learning Go Brrrr From First Principles* (blog, 2022) — the clearest explanation of the memory bandwidth problem in practice.

Benchmark to Run: Profile a GPT-2 forward pass with `torch.profiler`. Identify the top 5 kernels by wall time. For each, use `ncu` to get arithmetic intensity. Classify each as compute-bound or bandwidth-bound. This is the core skill of Chapter 7 — practice it on real code.

Months 7–8: Training and Serving Systems

Goal: You can design a distributed training system for a 70B parameter model from first principles, explain PagedAttention memory management, and derive KV cache memory requirements for a production serving deployment.

Chapters — Read Deeply:

- **Chapter 10** — Training Systems Architecture at Scale. 4D parallelism decision tree. ZeRO memory analysis. Pipeline bubble math. Fault tolerance at 1,000+ GPU scale.
- **Chapter 11** — KV Cache: The Heart of LLM Serving. PagedAttention deep dive. Fragmentation analysis. Prefix caching. KV cache memory sizing formula.

Hands-On Project: Build the memory budget for a 70B parameter model on a 8×H100 node. Compute: weight memory (BF16), optimizer states (Adam FP32), gradient memory, activation memory (with recomputation), and KV cache for 512-token context. The total should exceed 640GB — identify which parallelism strategy is needed to make it fit.

Papers to Read:

1. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism* (Shoeybi et al., 2019) — tensor and pipeline parallelism.

2. *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models* (Rajbhandari et al., 2020) — the three ZeRO stages from first principles.
3. *Efficient Memory Management for Large Language Model Serving with PagedAttention* (Kwon et al., 2023) — the paper that became vLLM.

Benchmark to Run: Run `vllm benchmark_throughput.py` with Llama-3-8B. Sweep batch size from 1 to 64. Record TTFT and throughput. Plot the prefill-decode regime transition. The point where throughput stops scaling with batch size is the memory bandwidth limit.

Months 9–10: Infrastructure at Scale

Goal: You can design the networking topology for a 1,024-GPU training cluster, specify storage systems for a petabyte-scale dataset pipeline, and build a fleet-wide telemetry stack that detects fail-slow GPUs.

Chapters — Read Deeply:

- **Chapter 14** — Networking and Collective Communication. InfiniBand topology. NCCL AllReduce algorithm selection. Bandwidth vs. latency regime classification. NCCL environment variable tuning.
- **Chapter 15** — Storage, Data Pipelines, and I/O Performance. Checkpoint storage sizing math. Dataset pipeline bottleneck analysis. Object store vs. parallel filesystem tradeoffs.

Hands-On Project: Run `nccl-tests` on a multi-GPU system. Measure `busbw` for AllReduce at message sizes from 1KB to 1GB. Identify the latency-dominated and bandwidth-dominated regimes. Plot the crossover. Compare measured busbw to the theoretical NVLink or InfiniBand bandwidth.

Papers to Read:

1. *Scaling Distributed Machine Learning with the Parameter Server* (Li et al., 2014) — why we moved away from parameter servers to AllReduce.
2. *Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour* (Goyal et al., 2017) — the distributed training scaling laws that still apply.
3. *BANDANA: Using Non-Volatile Memory for Storing Deep Learning Models* (Tang et al., 2019) — storage hierarchy reasoning for checkpointing.

Benchmark to Run: Measure checkpoint write bandwidth on your storage system. Time `torch.save(model.state_dict(), path)` for a 7B and 70B model. Compute the checkpoint storage rate in GB/s. Compare to the filesystem theoretical bandwidth. Identify whether checkpoint I/O is a training bottleneck for your cluster.

Months 11–12: Architecture Synthesis and Principal Identity

Goal: You operate at principal level. You can walk into an architecture review with quantitative analysis, drive a system design discussion, communicate tradeoffs in terms that move hardware decisions forward, and build the professional profile that makes you identifiable as a principal architect in AI infrastructure.

Chapters — Read Deeply:

- **Chapter 5** — Thermal Design, Power Management, and TCO. Full cluster TCO model. Power delivery constraints. $/token as the cost metric. Carbon and sustainability accounting.
- **Chapter 18** — The Principal Architect: Career, Interviews, and Professional Development. The four superpowers. STAR+Impact story framework. System design interview walkthroughs.

Hands-On Project: Build a complete, written architecture recommendation for a real or hypothetical system design decision. Example: "Should we use 256× H100 SXM or 512× H100 PCIe for a 70B training workload?" Include: FLOP budget, memory budget, communication analysis, TCO over 3 years, and a recommendation with explicit tradeoff labeling. Present it to a peer who will challenge it. Revise based on their challenges. The final document should be 3–5 pages and should be something you would be proud to send to a VP of Engineering.

Papers to Read:

1. *The Llama 3 Herd of Models* (Meta, 2024) — read the infrastructure section. This is what principal-level AI infrastructure engineering looks like in practice.
2. *DeepSeek-V3 Technical Report* (DeepSeek AI, 2024) — the MoE training infrastructure section is a graduate-level tutorial in Chapter 13 topics.
3. *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM* (Narayanan et al., 2021) — the 4D parallelism framework used throughout Chapter 10.

Benchmark to Run: Run the full LLM inference serving system design mock interview from §18.10. Timer set to 45 minutes. No notes. Record yourself if possible. Score yourself on: did you frame the problem before proposing a solution? Did you label every tradeoff? Did you produce back-of-envelope numbers? Did you connect design choices to measurable outcomes? This is the benchmark that matters.

Curriculum Summary

Months	Focus	Chapters	Key Project
1–2	Foundations	Ch 1, Ch 2	Transformer profiling from scratch
3–4	Hardware	Ch 3A, Ch 4	HBM bandwidth roofline measurement
5–6	Optimization	Ch 7, Ch 9	Triton fused kernel implementation
7–8	Systems	Ch 10, Ch 11	70B memory budget derivation
9–10	Infrastructure	Ch 14, Ch 15	nccl-tests + checkpoint bandwidth
11–12	Synthesis	Ch 5, Ch 18	Written architecture recommendation

---

Table of Contents

6 Parts · 18 Chapters · 3 Appendices · 450+ Pages

Front Matter

- Preface — Who This Book Is For, and What Makes It Different
- How to Use This Book — Four Reader Paths, Time-Budget Reference
- The 12-Month Principal-Level Curriculum
- Confidence Label Key and Code Block Conventions
- Table of Contents

Part I — Foundations

The performance mindset, the transformer architecture, and the analytical frameworks that unify the rest of the book

Chapter 1 — The AI/ML Performance Architecture Mindset 1.1 The Seven-Layer Performance Stack
1.2 Roofline Analysis: The Universal Performance Model

1.3 Arithmetic Intensity: The Master Classifier
1.4 The Three Performance Regimes
1.5 Hardware Efficiency: MFU and HFU
1.6 Measurement Discipline — Profiling Before Optimizing
1.7 The Performance Engineer's Hypothesis Loop
1.8 Scaling Laws and Their Performance Implications
1.9 Key Takeaways
Review Questions

Chapter 2 — The Transformer Architecture: A Performance Engineer's Deep Dive 2.1 The Transformer as a GEMM Machine
2.2 Tokenization and Embedding — Where Data Enters the System
2.3 Multi-Head Self-Attention — Derivation and Arithmetic
2.4 Grouped-Query and Multi-Query Attention
2.5 Rotary Position Encoding (RoPE)
2.6 Feed-Forward Networks and the Role of Activation Functions
2.7 Layer Normalization: RMSNorm, LayerNorm, and BatchNorm
2.8 Residual Connections and Gradient Flow
2.9 The Decoder-Only Architecture (GPT Family)
2.10 FLOP Budget Analysis for Major Model Families
2.11 Memory Footprint Analysis
2.12 Key Takeaways
Review Questions

Part II — Hardware Deep Dive

GPU architecture, memory systems, and the physical constraints that bound every optimization

Chapter 3A — GPU and Accelerator Architecture Fundamentals 3A.1 The GPU Execution Model: SMs, Warps, and Threads
3A.2 Tensor Cores: The Inner Loop of AI Compute
3A.3 Memory Hierarchy: Registers, L1, L2, HBM
3A.4 NVLink and NVSwitch: The Intra-Node Fabric
3A.5 SXM vs. PCIe — Packaging Implications
3A.6 TPUs, Gaudi, and Custom Silicon — Comparative Analysis
3A.7 Choosing the Right Accelerator for Your Workload
3A.8 Key Takeaways
Review Questions

Chapter 3B — GPU Architecture Roadmap: NVIDIA and AMD Generations 3B.1 NVIDIA: Ampere → Hopper → Blackwell → Rubin
3B.2 AMD: CDNA1 → CDNA2 → CDNA3 (MI300X) → CDNA4

3B.3 Interconnect Evolution: NVLink 4.0, NVLink 5.0, Infinity Fabric
3B.4 Memory Technology Evolution: HBM2e → HBM3 → HBM3e
3B.5 Reading the Hardware Roadmap: What Changes and What Does Not
3B.6 Key Takeaways
Review Questions

Chapter 4 — Memory Hierarchy and HBM Deep Dive 4.1 Why Memory Is the Bottleneck in AI
4.2 HBM Physical Architecture: Stacks, Dies, and Channels
4.3 HBM Bandwidth: Theory vs. Sustained Reality
4.4 ECC, Error Rates, and Reliability Engineering
4.5 The DRAM Roofline: Deriving Memory-Bound Performance Limits
4.6 Cache Hierarchy in Detail: L1, L2, Shared Memory
4.7 Memory Access Pattern Taxonomy
4.8 Memory Capacity Planning for Large Models
4.9 Key Takeaways
Review Questions

Chapter 5 — Thermal Design, Power Management, and TCO 5.1 GPU Power Delivery: TDP, TBP, and Sustained Power
5.2 Thermal Resistance and Junction Temperature
5.3 Cooling Architecture: Air, Direct Liquid, Immersion
5.4 Power Throttling and Performance Cliffs
5.5 Rack-Level Power Budget and PDU Design
5.6 TCO Model: Capex, Opex, Power, Cooling
5.7 $/Token: The Operational Cost Metric
5.8 Carbon Accounting for AI Workloads
5.9 Key Takeaways
Review Questions

Part III — Optimization Techniques

The algorithms, kernels, and compiler tools that close the gap between hardware limits and application performance

Chapter 6 — Inference Systems Architecture 6.1 The Two Regimes: Prefill and Decode
6.2 Continuous Batching and Iteration-Level Scheduling
6.3 Chunked Prefill and Disaggregated Serving
6.4 Throughput-Latency Curves and SLO Budget Allocation
6.5 TTFT and TPOT: The Two Serving Metrics That Matter
6.6 Speculative Decoding at the Serving Layer
6.7 Tensor Parallelism in Inference
6.8 Serving Frameworks: vLLM, TensorRT-LLM, SGLang

6.9 Multi-Modal Serving Architecture
6.10 Observability in the Serving Stack
6.11 The Tokens-Per-Second Optimization Hierarchy
6.12 Multi-Tenant Serving: Cost Model and Isolation
6.13 Worked Example: Optimizing a Production vLLM Deployment
6.14 Key Takeaways
Review Questions

Chapter 7 — GPU Kernels and CUDA Optimization 7.1 The Kernel Execution Model
7.2 Memory Coalescing: The Most Important GPU Optimization
7.3 Shared Memory and Bank Conflicts
7.4 Warp Divergence and Occupancy
7.5 Tensor Core Programming: WMMA and cuBLAS
7.6 FlashAttention: The IO-Aware Kernel Design
7.7 FlashAttention-2 and FlashAttention-3 Improvements
7.8 Triton: Writing Custom Kernels Without CUDA
7.9 Kernel Profiling: ncu Counters and Their Meanings
7.10 Key Takeaways
Review Questions

Chapter 8 — Quantization 8.1 Why Quantization Works: The Numeric Headroom Argument
8.2 INT8 Quantization: Symmetric and Asymmetric
8.3 FP8: The Training and Inference Sweet Spot
8.4 GPTQ, AWQ, and Post-Training Quantization Algorithms
8.5 QAT: Quantization-Aware Training
8.6 KV Cache Quantization
8.7 Quantization-Induced Accuracy Degradation: Diagnosis and Recovery
8.8 The Production Quantization Decision Framework (Step 0–5)
8.9 Quantization Decision Table: Model × Hardware × Accuracy Regime
8.10 Key Takeaways
Review Questions

Chapter 9 — Operator Fusion and Compiler Optimization 9.1 Why Fusion Exists: The Memory Traffic Argument
9.2 Identifying Fusion Candidates
9.3 PyTorch 2.0 and `torch.compile`
9.4 Triton: Composable Custom Kernels
9.5 XLA and JAX: Fusion in a Functional Framework
9.6 TensorRT: Deployment-Optimized Fusion
9.7 FlashAttention as the Canonical Fused Kernel
9.8 Compile Failures: Diagnosis and Fallback Strategies
9.9 Key Takeaways
Review Questions

Part IV — Training and Serving Systems

The distributed systems architectures that scale AI from a single GPU to tens of thousands

Chapter 10 — Training Systems Architecture at Scale 10.1 The Training Stack: Framework, Collective, Scheduler, Storage
10.2 Data Parallelism and Gradient Synchronization
10.3 Tensor Parallelism: Megatron-Style Column/Row Sharding
10.4 Pipeline Parallelism and the Bubble Problem
10.5 Sequence Parallelism and Context Parallelism
10.6 The 4D Parallelism Decision Tree
10.7 ZeRO: Memory Optimization Through Sharding
10.8 Gradient Checkpointing and Recomputation Strategy
10.9 Training Job Failure Taxonomy and Recovery
10.10 Training Experiment Tracking and Observability
10.11 Key Takeaways
Review Questions

Chapter 11 — KV Cache: The Heart of LLM Serving 11.1 The KV Cache Memory Problem
11.1.1 KV Cache Memory Sizing Formula
11.1.2 KV Cache vs. Weight Memory Budget
11.1.3 How Context Length Explodes Memory
11.1.4 GQA/MQA Memory Reduction
11.1.5 Worked Example: 70B Model at Production Scale
11.1.6 Fragmentation Analysis and Page Size Selection
11.2 PagedAttention: Virtual Memory for KV Cache
11.3 Prefix Caching and Prompt Sharing
11.4 KV Cache Quantization
11.5 Multi-Level KV Cache Tiering
11.6 KV Cache in Multi-GPU Deployments
11.7 Key Takeaways
Review Questions

Chapter 12 — Benchmarking and Performance Measurement 12.1 Benchmark Design Principles
12.2 MLPerf Inference: Methodology and Interpretation
12.3 MLPerf Training: Methodology and Interpretation
12.4 Microbenchmarks: What They Measure and What They Misrepresent
12.5 System-Level Benchmarking: End-to-End Metrics
12.6 Regression Detection: Statistical Methods
12.7 Capacity Planning From Benchmark Data

12.8 The Benchmark Harness Architecture
12.9 Regression CI Pipeline Design
12.10 Fleet Health Monitoring with DCGM
12.11 Key Takeaways
Review Questions

Part V — Infrastructure at Scale

The networking, storage, cluster management, and observability systems that make AI at scale possible

Chapter 13 — Speculative Decoding and MoE Infrastructure 13.1 Speculative Decoding: Theory and Acceptance Math
13.2 Speculative Decoding Profitability Analysis
13.3 Draft Model Selection and Colocation
13.4 Mixture-of-Experts Architecture
13.5 MoE Routing: Top-K, Expert Choice, Switch Transformer
13.6 Expert Parallelism and All-to-All Communication
13.7 Load Imbalance: Diagnosis and Mitigation
13.8 MoE Infrastructure Design at Scale
13.9 Key Takeaways
Review Questions

Chapter 14 — Networking and Collective Communication 14.1 The Communication Bottleneck in Distributed AI
14.2 InfiniBand Architecture: HDR, NDR, XDR
14.3 RDMA and the Bypass Model
14.4 NCCL: Architecture and Algorithm Selection
14.5 AllReduce Algorithms: Ring, Tree, Recursive Halving
14.6 NCCL Environment Variable Tuning — Top 10
14.7 Network Topology Sizing: Math-Driven Design
14.8 Measurement Workflow: Deploy → Configure → Validate
14.9 Ethernet for AI: RoCEv2 and DCQCN
14.10 Key Takeaways
Review Questions

Chapter 15 — Storage, Data Pipelines, and I/O Performance 15.1 Storage Hierarchy for AI Workloads
15.2 Dataset Pipeline Architecture
15.3 Data Stalls: Detection and Elimination
15.4 Sequence Packing for Training Efficiency
15.5 Checkpoint Storage: Sizing, Frequency, and Format

15.6 Parallel Filesystem vs. Object Store
15.7 KV Cache Tiering to Storage
15.8 Storage Specification Framework
15.9 Key Takeaways
Review Questions

Chapter 16A — Cluster Management and Multi-Tenant Systems 16A.1 SLURM vs. Kubernetes for AI Workloads
16A.2 Gang Scheduling: Why AI Requires All-or-Nothing Allocation
16A.3 Multi-Tenant Isolation: MIG, vGPU, Namespace Partitioning
16A.4 SLA Enforcement and Priority Queues
16A.5 Preemption Strategies and Checkpointing Integration
16A.6 Cluster Utilization Metrics and Efficiency Reporting
16A.7 Autoscaling for Inference Workloads
16A.8 Key Takeaways
Review Questions

Chapter 17 — Telemetry, Observability, and Profiling Tools 17.1 The Observability Stack for AI Infrastructure
17.2 Prometheus and Grafana: The Fleet Monitoring Layer
17.3 DCGM: GPU Health and Performance Telemetry
17.4 Nsight Systems: System-Level Timeline Profiling
17.5 Nsight Compute: Kernel-Level Counter Analysis
17.6 ROCm Omniperf: AMD Equivalent Profiling
17.7 The Tool Selection Decision Tree
17.8 SLA Regression Diagnosis Workflow
17.9 Fail-Slow GPU Detection
17.10 CI Pipeline for Performance Regression
17.11 Key Takeaways
Review Questions

Part VI — Principal Career

The identity, skills, stories, and system design patterns of the AI/ML Principal Performance Architect

Chapter 18 — The Principal Architect: Career, Interviews, and Professional Development 18.0 The $1M Question — What Makes This Role Different
18.1 What "Principal" Means in AI Infrastructure
18.2 The Four Superpowers of the Principal AI Performance Architect
18.3 Breadth vs. Depth — Positioning for Principal Roles
18.4 The STAR+Impact Story Framework

18.5 Story Bank — Real STAR Examples
18.6 The Principal-Level Differentiating Question
18.7 Professional Communication Strategy
18.8 Building the Principal Profile — The Five Moves
18.9 System Design Interview Playbook
18.10 System Design Walkthrough 1 — LLM Inference Serving at Scale
18.11 System Design Walkthrough 2 — Distributed LLM Training System
18.12 Reading Roadmap — Papers, Tools, and Continued Growth
18.13 Key Takeaways

Back Matter

Appendix A — Hardware Reference Tables

- GPU Specification Comparison: A100 · H100 · H200 · MI300X · GB200
- Memory Bandwidth and Capacity by Generation
- NVLink and NVSwitch Topology Reference
- InfiniBand Bandwidth: HDR · NDR · XDR
- Roofline Ridge Points by GPU Model

Appendix B — Profiling Tool Quick Reference

- Tool Selection Decision Tree
- Nsight Systems: Key Metrics and Workflow
- Nsight Compute: Essential Counters
- DCGM: Field IDs and Health Monitoring
- NCCL Debug Environment Variables
- nccl-tests: busbw Interpretation Reference

Appendix C — Glossary of Key Terms *~80 terms from Arithmetic Intensity to ZeRO-3*

AI/ML Infrastructure from Silicon to Scale — First Edition · 2026
Venkat Vinjam · AI/ML Performance Architect · AMD | Ex-Intel
© 2026 Venkat Vinjam. All Rights Reserved.

"Every optimization decision in AI/ML infrastructure is a question of arithmetic intensity, memory bandwidth, and communication volume — master these three,

and you can reason about any system."